

# Digital Electronics

## Sessional Exam - Detailed Solutions

### Q1. Explain in details Multiplexer with simple circuit.

**Definition:** A Multiplexer (often abbreviated as MUX) is a combinational logic circuit that selects one of many analog or digital input signals and forwards the selected input into a single output line. It acts as a digital switch or data selector.

**Principle of Operation:** A multiplexer has  $2^n$  data input lines,  $n$  selection lines, and exactly 1 output line. The selection lines determine which specific data input is connected to the output.

#### Example: 2-to-1 Multiplexer

The simplest multiplexer is the 2-to-1 MUX.

- **Inputs:** Two data lines ( $D_0$  and  $D_1$ )
- **Selection Line:** One select line ( $S$ ) because  $2^1 = 2$ .
- **Output:** One output line ( $Y$ )

#### Truth Table:

| Select Line (S) | Output (Y) |
|-----------------|------------|
| 0               | $D_0$      |
| 1               | $D_1$      |

*Explanation:* When  $S = 0$ , the upper input  $D_0$  is routed to the output  $Y$ . When  $S = 1$ , the lower input  $D_1$  is routed to the output  $Y$ .

#### Boolean Expression:

From the truth table, the logical expression for the output  $Y$  is:

$$Y = \bar{S} \cdot D_0 + S \cdot D_1$$

Simple Circuit Description (Logic Gates):

To construct this circuit, you need:

- 1. **One NOT gate:** To invert the select line S to get  $\overline{S}$ .
- 2. **Two AND gates:**
  - The first AND gate takes  $\overline{S}$  and  $D_0$  as inputs.
  - The second AND gate takes S and  $D_1$  as inputs.
- 3. **One OR gate:** Takes the outputs of both AND gates to produce the final output Y.

Q2. Draw & explain Nand gate circuit with truth table.

**Definition:** A NAND gate (NOT-AND) is a universal logic gate. It operates as an AND gate followed by a NOT gate. It produces a LOW (0) output only if all its inputs are HIGH (1). For any other combination of inputs, the output is HIGH (1).

It is called a "Universal Gate" because you can build any other basic logic gate (AND, OR, NOT) using only NAND gates.

**Symbol representation:** The standard symbol for a NAND gate is the D-shaped AND gate symbol with a small circle (inversion bubble) at the output end.

Boolean Expression:

If the inputs are A and B, the output Y is the complement of their logical product:

$$Y = \overline{A \cdot B}$$

Truth Table (2-Input NAND Gate):

| Input A | Input B | Output Y |
|---------|---------|----------|
| 0       | 0       | 1        |
| 0       | 1       | 1        |
| 1       | 0       | 1        |
| 1       | 1       | 0        |

**Explanation:**

- When both A and B are 0, the AND operation gives 0. Inverting it gives 1.
- When one input is 0 and the other is 1, the AND operation gives 0. Inverting it gives 1.
- When both A and B are 1, the AND operation gives 1. Inverting it gives 0.

Q3. Draw & explain R S flip flop with TT.

**Definition:** An SR (Set-Reset) flip-flop is the most basic sequential logic circuit. It is a 1-bit memory bistable device that has two stable states. It is built using cross-coupled NOR gates or NAND gates.

Below is the explanation for an Active-High SR Flip-Flop using NOR gates.

Inputs and Outputs:

- **S (Set):** Sets the output Q to 1.
- **R (Reset):** Resets the output Q to 0.
- **Outputs:** Normal output Q and inverted output  $\overline{Q}$ .

Truth Table:

Here,  $Q_n$  represents the present state, and  $Q_{n+1}$  represents the next state.

| S<br>(Set) | R<br>(Reset) | Output<br>$Q_{n+1}$ | State / Action                           |
|------------|--------------|---------------------|------------------------------------------|
| 0          | 0            | $Q_n$               | <b>Hold:</b> Output remains unchanged.   |
| 0          | 1            | 0                   | <b>Reset:</b> Output is forced to 0.     |
| 1          | 0            | 1                   | <b>Set:</b> Output is forced to 1.       |
| 1          | 1            | Invalid             | <b>Forbidden:</b> Violates output rules. |

Circuit Construction (NOR gate based):

The circuit consists of two NOR gates.

- The output of the first NOR gate ( $\overline{Q}$ ) is fed back as an input to the second NOR gate.
- The output of the second NOR gate (Q) is fed back as an input to the first NOR gate.
- The free input of the first NOR gate is R (Reset).
- The free input of the second NOR gate is S (Set).

Q4. Draw & explain half adder & full adder with TT.

1. Half Adder

**Definition:** A half adder is a combinational circuit that performs the arithmetic addition of two 1-bit binary numbers (A and B). It produces two outputs: Sum (S) and Carry (C).

Boolean Expressions:

- Sum (S):  $S = A \oplus B$  (A XOR B)
- Carry (C):  $C = A \cdot B$  (A AND B)

Truth Table:

| A | B | Sum (S) | Carry (C) |
|---|---|---------|-----------|
| 0 | 0 | 0       | 0         |
| 0 | 1 | 1       | 0         |
| 1 | 0 | 1       | 0         |
| 1 | 1 | 0       | 1         |

**Circuit Explanation:** It consists of two logic gates connected in parallel to the inputs A and B. An XOR gate produces the Sum output, and an AND gate produces the Carry output.

2. Full Adder

**Definition:** A half adder has a major limitation: it cannot accept a carry bit from a previous lower-order addition. A full adder overcomes this by adding three 1-bit binary numbers: two data inputs (A and B) and one Carry-in ( $C_{in}$ ) from a previous stage.

**Outputs:** It produces a Sum (S) and a Carry-out ( $C_{out}$ ).

Boolean Expressions:

- Sum (S):  $S = A \oplus B \oplus C_{in}$
- Carry-out ( $C_{out}$ ):  $C_{out} = (A \cdot B) + C_{in}(A \oplus B)$

**Truth Table:**

| A | B | C <sub>in</sub> | Sum (S) | Carry Out (C <sub>out</sub> ) |
|---|---|-----------------|---------|-------------------------------|
| 0 | 0 | 0               | 0       | 0                             |
| 0 | 0 | 1               | 1       | 0                             |
| 0 | 1 | 0               | 1       | 0                             |
| 0 | 1 | 1               | 0       | 1                             |
| 1 | 0 | 0               | 1       | 0                             |
| 1 | 0 | 1               | 0       | 1                             |
| 1 | 1 | 0               | 0       | 1                             |
| 1 | 1 | 1               | 1       | 1                             |

**Circuit Explanation:** A full adder can be constructed using two Half Adders and one OR gate.

1. The first Half Adder adds A and B.
2. The second Half Adder adds the Sum of the first Half Adder to C<sub>in</sub> to produce the final Sum (S).
3. The Carry outputs from both Half Adders are fed into an OR gate to produce the final Carry-out (C<sub>out</sub>).